

Session 2: Data Import, Cleaning, and Exploratory Data Analysis (EDA)

Compiled by D K. Muriithi

1. Data Importation

Data importation is the first step in any data analysis workflow. In Python, the **pandas** library is widely used for loading data from various sources, such as CSV, Excel, SQL databases, JSON, and even web scraping. Proper data importation ensures that the dataset is structured correctly and is ready for further preprocessing. This comprehensive example demonstrates:

- **CSV Files:** The most common format for storing tabular data, where values are separated by commas.
- **Excel Files:** Useful for structured data with multiple sheets. Using pandas' `read_excel()`.
- **JSON and XML Files:** Common in web applications and APIs.
- **SQL Databases**

When importing data, it is important to inspect the structure to confirm the correct delimiter, encoding, and column names. Errors in data importation, such as incorrect data types or encoding mismatches, may require additional preprocessing.

2. Data Cleaning

Real-world datasets often contain missing values, duplicates, and inconsistencies that must be addressed before analysis. Data cleaning ensures that the dataset is structured correctly and free from errors that might affect the accuracy of the results.

Handling Missing Data

Missing values can occur due to data entry errors, incomplete records, or system issues. There are several ways to handle missing data:

- **Deletion:** Removing rows or columns with missing values if they are minimal.
- **Imputation:** Replacing missing values with statistical measures such as the mean, median, or mode.
- **Forward or Backward Fill:** Filling missing values using previous or next available values in time-series data.

Handling Outliers

Outliers are extreme values that differ significantly from the rest of the data and can distort analysis results. They can be detected using statistical methods such as:

- **Z-score method:** Identifies data points that are several standard deviations away from the mean.
- **Interquartile Range (IQR) method:** Identifies outliers based on quartiles.
- **Visualization methods:** Box plots and scatter plots help in detecting extreme values.

3. Summary Statistics

Summary statistics provide insights into the distribution, central tendency, and spread of the data. Some of the key statistical measures include:

- **Mean:** The average value of a dataset.
- **Median:** The middle value when the data is sorted.
- **Mode:** The most frequently occurring value.
- **Variance and Standard Deviation:** Measures the spread of data around the mean.
- **Skewness and Kurtosis:** Used to understand the shape of the distribution.

These statistics help analysts understand the nature of the dataset and whether further transformations are necessary before modeling.

4. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of analyzing datasets to summarize their key characteristics, often with the help of visualizations. It helps to identify patterns, trends, and potential issues within the data.

Data Visualization

Visualization is a key part of EDA, as it provides an intuitive understanding of data relationships. Common types of visualizations include:

- **Histograms:** Show the distribution of numerical data.
- **Box Plots:** Identify outliers and the spread of data.
- **Scatter Plots:** Show relationships between two numerical variables.
- **Bar Charts:** Compare categorical data.
- **Heatmaps:** Display correlations between multiple numerical variables.

Using **Matplotlib** and **Seaborn**, analysts can create these visualizations to better understand the data before applying statistical or machine learning models.

5. Importance of EDA

EDA is a critical step before building models, as it helps in:

- Understanding the data structure and identifying inconsistencies.
- Detecting missing values, outliers, and unusual patterns.
- Selecting appropriate features for predictive modeling. Improving data preprocessing and transformation steps.
- Summarize key characteristics of a dataset.

- Improving data preprocessing and transformation steps.

By the end of this session, learners will be able to import datasets, clean data by handling missing values and outliers, compute summary statistics, and create visualizations for exploratory data analysis.

2. Removing Duplicates

```
python

# Detect duplicates
df.duplicated().sum()          # count duplicate rows

# Remove duplicates
df.drop_duplicates()           # remove all duplicate rows
df.drop_duplicates(subset=['email']) # based on specific column
df.drop_duplicates(keep='last')    # keep last occurrence
```

3. Fixing Data Types

```
python

# Check data types
df.dtypes

# Convert types
df['age'] = df['age'].astype(int)
df['price'] = df['price'].astype(float)
df['date'] = pd.to_datetime(df['date']) # convert to datetime
df['is_active'] = df['is_active'].astype(bool)
```

4. Renaming & Dropping Columns

```
python

# Rename columns
df.rename(columns={'old_name': 'new_name', 'qty': 'quantity'}, inplace=True)

# Lowercase all column names
df.columns = df.columns.str.lower()

# Remove spaces in column names
df.columns = df.columns.str.replace(' ', '_')

# Drop unnecessary columns
df.drop(columns=['irrelevant_col', 'temp'], inplace=True)
```

5. Handling Outliers

```
python

import numpy as np

# Using Z-score
from scipy import stats
z_scores = stats.zscore(df['salary'])
df = df[(abs(z_scores) < 3)] # keep rows within 3 std deviations

# Using IQR (Interquartile Range)
Q1 = df['salary'].quantile(0.25)
Q3 = df['salary'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['salary'] >= Q1 - 1.5 * IQR) &
        (df['salary'] <= Q3 + 1.5 * IQR)]
```

6. Cleaning String / Text Data

```
python

# Remove whitespace
df['name'] = df['name'].str.strip()

# Change case
df['name'] = df['name'].str.lower()
df['name'] = df['name'].str.title()

# Remove special characters
df['phone'] = df['phone'].str.replace(r'\D', '', regex=True)

# Extract pattern using regex
df['year'] = df['date_str'].str.extract(r'(\d{4})')
```

7. Standardizing / Normalizing Data

```
python

from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Normalization (0 to 1)
scaler = MinMaxScaler()
df['age_scaled'] = scaler.fit_transform(df[['age']])

# Standardization (mean=0, std=1)
scaler = StandardScaler()
df['salary_scaled'] = scaler.fit_transform(df[['salary']])
```

8. Encoding Categorical Data

```
python

# Label encoding
df['gender'] = df['gender'].map({'Male': 0, 'Female': 1})

# One-hot encoding
df = pd.get_dummies(df, columns=['city'], drop_first=True)

# Using sklearn
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['category'] = le.fit_transform(df['category'])
```

Complete Cleaning Workflow

```
python

import pandas as pd

df = pd.read_csv('data.csv')

# Step 1 - Inspect
print(df.shape)
print(df.info())
print(df.isnull().sum())

# Step 2 - Clean
df.drop_duplicates(inplace=True)
df.dropna(subset=['id', 'name'], inplace=True)
df['age'].fillna(df['age'].median(), inplace=True)
df.columns = df.columns.str.lower().str.replace(' ', '_')
df['date'] = pd.to_datetime(df['date'])
df['name'] = df['name'].str.strip().str.title()

# Step 3 - Verify
print(df.isnull().sum())
print(df.dtypes)
print(df.head())
```

Summary Table

Problem	Solution
Missing values	<code>fillna()</code> , <code>dropna()</code>
Duplicates	<code>drop_duplicates()</code>
Wrong data types	<code>astype()</code> , <code>to_datetime()</code>
Outliers	Z-score, IQR method
Messy strings	<code>str.strip()</code> , <code>str.lower()</code> , regex
Categorical data	<code>map()</code> , <code>get_dummies()</code>
Inconsistent scaling	<code>MinMaxScaler</code> , <code>StandardScaler</code>

Data cleaning typically takes **60–80% of a data project's time** — clean data leads to better analysis and more accurate models!

Data Importations

```
In [1]: # Import pandas library for data manipulation and analysis
import pandas as pd
```

```
In [2]: data = pd.read_csv("GSSsubset.csv") # Load the CSV file containing GSS data into a
data.head()                                # Display the first 5 rows of the dataset to get
```

```
Out[2]:
```

	id	sex	degree	income	marital	age	height	weight	hrrwrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6

```
In [3]: data.tail() # Display the last 5 rows of the dataframe to inspect the data
```

```
Out[3]:
```

	id	sex	degree	income	marital	age	height	weight	hrrwrk
989	2531	MALE	HIGH SCHOOL	1478.00	NEVER MARRIED	40	71	230	48
990	2535	MALE	HIGH SCHOOL	33255.00	DIVORCED	56	72	195	46
991	2536	MALE	HIGH SCHOOL	8313.75	NEVER MARRIED	24	68	145	40
992	2537	MALE	HIGH SCHOOL	27712.50	NEVER MARRIED	27	68	180	40
993	2538	FEMALE	HIGH SCHOOL	15703.75	WIDOWED	71	63	140	48

Data Cleaning and Preprocessing

Handling Missing Values

Check for Missing Values in Every Column

```
In [4]: missing_values = data.isnull().sum()

print("Missing values in each column:\n", missing_values)
```

Missing values in each column:

id	0
sex	0
degree	0
income	0
marital	0
age	0
height	0
weight	0
hrswrk	0

dtype: int64

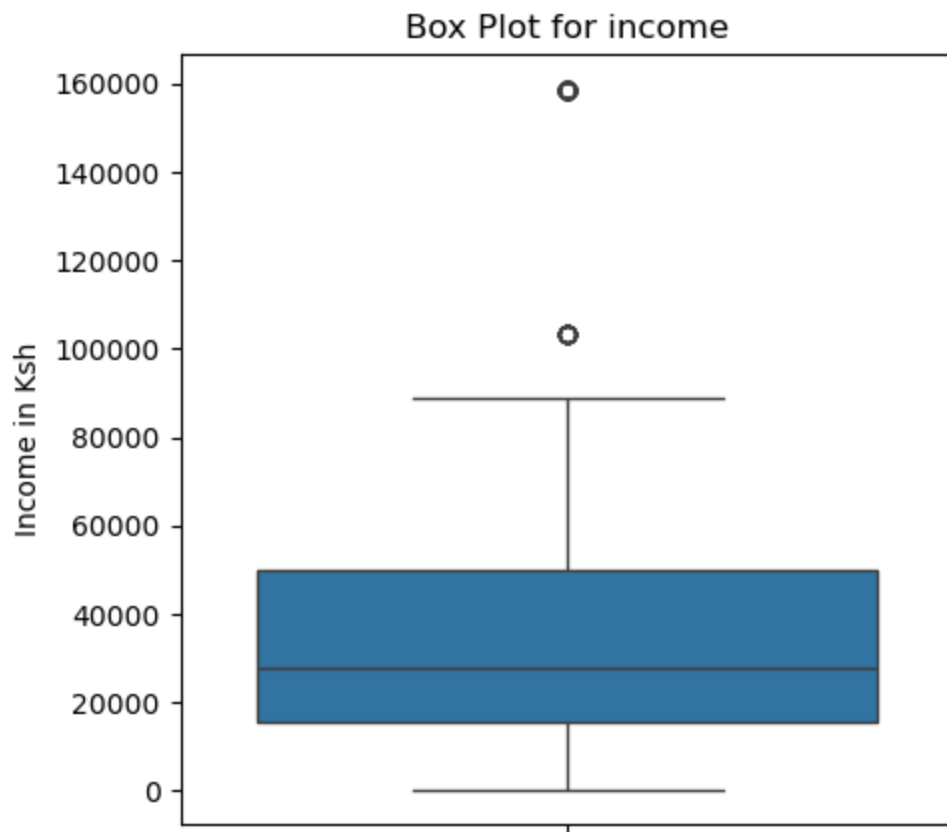
Remove rows with missing values

```
In [5]: data.dropna(inplace=True)
```

Identification of Outliers Using BoxPlots

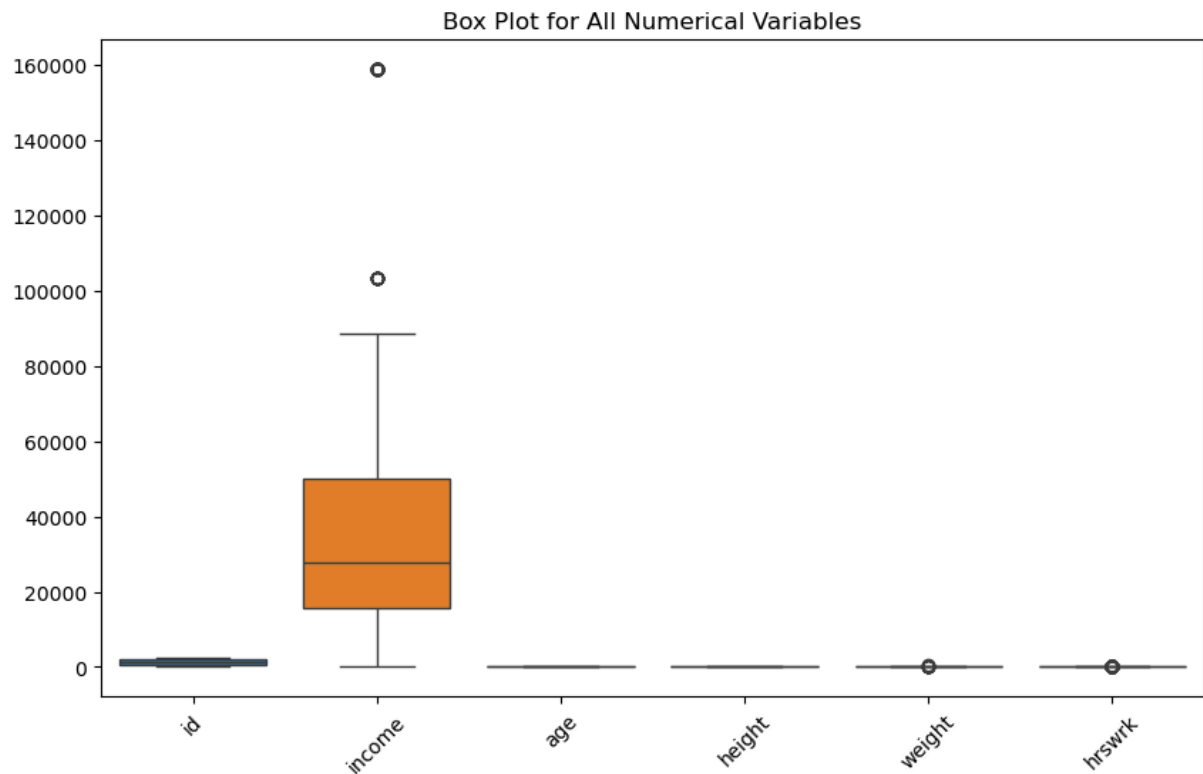
```
In [6]: #!pip install matplotlib  
#!pip install seaborn
```

```
In [7]: import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Create a box plot for income distribution  
plt.figure(figsize=(5, 5))  
sns.boxplot(data=data, y="income")  
  
# Customize plot appearance  
plt.title("Box Plot for income")  
plt.ylabel("Income in Ksh")  
plt.show() # Display the plot
```

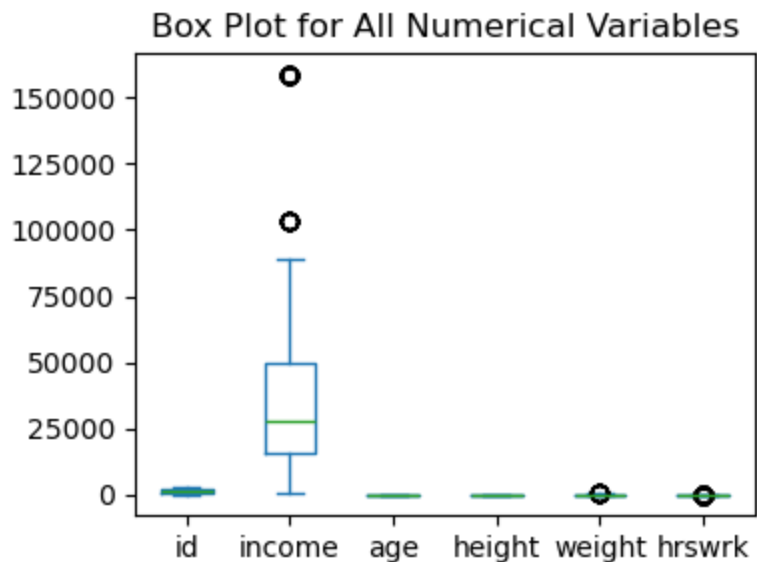
```
In [8]: # Create box plots for all numerical variables
plt.figure(figsize=(10, 6))
sns.boxplot(data=data)

# Customize plot
plt.xticks(rotation=45) # Rotate x-axis labels if needed
plt.title("Box Plot for All Numerical Variables")
plt.show()
```



```
In [9]: # Alternative: Using Pandas
# Create a box plot for all numerical variables in the dataset using pandas plotting

data.plot(kind='box', figsize=(4, 3))
plt.title("Box Plot for All Numerical Variables")
plt.show() # Display the plot
```



Removing Outliers

1. 1.5 Inter-Quartile Rule

```
In [10]: import pandas as pd

# Load dataset from CSV file
data = pd.read_csv("GSSsubset.csv")

# Calculate the Interquartile Range (IQR) for the income column
Q1 = data["income"].quantile(0.25) # First quartile (25th percentile)
Q3 = data["income"].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Calculate IQR as the difference between Q3 and Q1

# Define lower and upper bounds for outlier detection using the 1.5 * IQR rule
lower_bound = Q1 - 1.5 * IQR # Values below this are considered outliers
upper_bound = Q3 + 1.5 * IQR # Values above this are considered outliers

# Filter the dataset to remove outliers by keeping only values within bounds
data = data[(data["income"] >= lower_bound) & (data["income"] <= upper_bound)]

print("Outliers removed successfully!")
```

Outliers removed successfully!

2.Capping (Winsorization)

Instead of removing outliers, extreme values can be capped to a predefined threshold (e.g., replacing extreme values with the 5th and 95th percentiles).

```
In [11]: import numpy as np

# Define lower and upper caps
lower_cap = data["income"].quantile(0.05) # 5th percentile
upper_cap = data["income"].quantile(0.95) # 95th percentile

# Apply capping
data["income"] = np.where(data["income"] < lower_cap, lower_cap, data["income"])
data["income"] = np.where(data["income"] > upper_cap, upper_cap, data["income"])

print("Outliers have been capped!")
```

Outliers have been capped!

```
In [12]: import numpy as np

# Define lower and upper caps
lower_cap = data["income"].quantile(0.05) # 5th percentile
upper_cap = data["income"].quantile(0.95) # 95th percentile

# Apply capping
data["income"] = np.where(data["income"] < lower_cap, lower_cap, data["income"]) #
data["income"] = np.where(data["income"] > upper_cap, upper_cap, data["income"]) #

print("Outliers have been capped!")
```

Outliers have been capped!

Summary Statistics

```
In [13]: # Generate summary statistics for all numerical columns
# This uses pandas' describe() method which returns count, mean, std, min, 25%, 50%
summary_stats = data.describe()
# Display the summary statistics
# In Jupyter, the last line without assignment will be displayed as output
summary_stats
```

```
Out[13]:
```

	id	income	age	height	weight	hrrwrk
count	933.000000	933.000000	933.000000	933.000000	933.000000	933.000000
mean	1285.545552	29484.951501	43.981779	67.251876	180.096463	41.840300
std	731.089443	19560.571462	13.142365	4.028041	41.563687	13.918648
min	1.000000	3325.500000	19.000000	57.000000	90.000000	1.000000
25%	681.000000	15703.750000	33.000000	64.000000	150.000000	37.000000
50%	1272.000000	24017.500000	43.000000	67.000000	175.000000	40.000000
75%	1923.000000	40645.000000	54.000000	70.000000	202.000000	50.000000
max	2538.000000	73900.000000	79.000000	78.000000	410.000000	89.000000

For Categorical Variables

```
In [14]: # Assistant
import pandas as pd

# Summary statistics for categorical columns
cat_summary = data.describe(include=['O']) # 'O' stands for object (categorical data)
# This generates statistics like count,
# top (most frequent value), and frequency

cat_summary # Display the summary statistics for categorical columns
```

```
Out[14]:
```

	sex	degree	marital
count	933	933	933
unique	2	5	5
top	FEMALE	HIGH SCHOOL	MARRIED
freq	478	477	436

Obtain Summary Statistics of Choice

```
In [15]: import pandas as pd

# Load dataset from CSV file
df = pd.read_csv("GSSsubset.csv")
```

```

# Select only numeric columns (automatically excludes categorical variables)
# This helps focus analysis on quantitative data
df_numeric = df.select_dtypes(include=['number'])
## Remove the id Column as it's not relevant for statistical analysis
df_numeric = df_numeric.drop(columns=["id"])

# Compute comprehensive summary statistics for all numeric columns
summary_stats = {
    "Mean": df_numeric.mean(),          # Average value
    "Median": df_numeric.median(),      # Middle value (50th percentile)
    "Mode": df_numeric.mode().iloc[0], # Most frequent value (taking first if multi
    "Variance": df_numeric.var(),       # Measure of data spread
    "Standard Deviation": df_numeric.std(), # Square root of variance
    "Min": df_numeric.min(),            # Minimum value
    "Max": df_numeric.max(),            # Maximum value
    "Skewness": df_numeric.skew(),      # Measure of distribution asymmetry
    "Kurtosis": df_numeric.kurt()       # Measure of distribution "tailedness"
}
# Convert dictionary of statistics to DataFrame for better visualization and readability
summary_df = pd.DataFrame(summary_stats)

# Display the summary statistics table
summary_df

```

Out[15]:

	Mean	Median	Mode	Variance	Standard Deviation	Min	Max	Sk
income	36887.218352	27712.5	33255.0	1.204247e+09	34702.256068	369.5	158656.952	2
age	44.488934	44.0	53.0	1.723146e+02	13.126865	19.0	79.000	0
height	67.414487	67.0	70.0	1.630335e+01	4.037741	57.0	79.000	0
weight	181.318913	175.0	180.0	1.734024e+03	41.641615	90.0	410.000	0
hrswrk	42.640845	40.0	40.0	2.072153e+02	14.394974	1.0	89.000	0

Data Visualization

Histogram

```

In [16]: import matplotlib.pyplot as plt
import seaborn as sns

# Set figure size for better visualization
plt.figure(figsize=(6, 4))

# Create histogram using Seaborn with KDE (Kernel Density Estimate) overlay
# - data=df: Uses the dataframe 'df' as the data source
# - x="income": Plots the 'income' column from the dataframe
# - bins=10: Divides the data into 10 equal-width bins
# - kde=True: Adds a kernel density estimate curve
# - color="red": Sets the histogram color to red

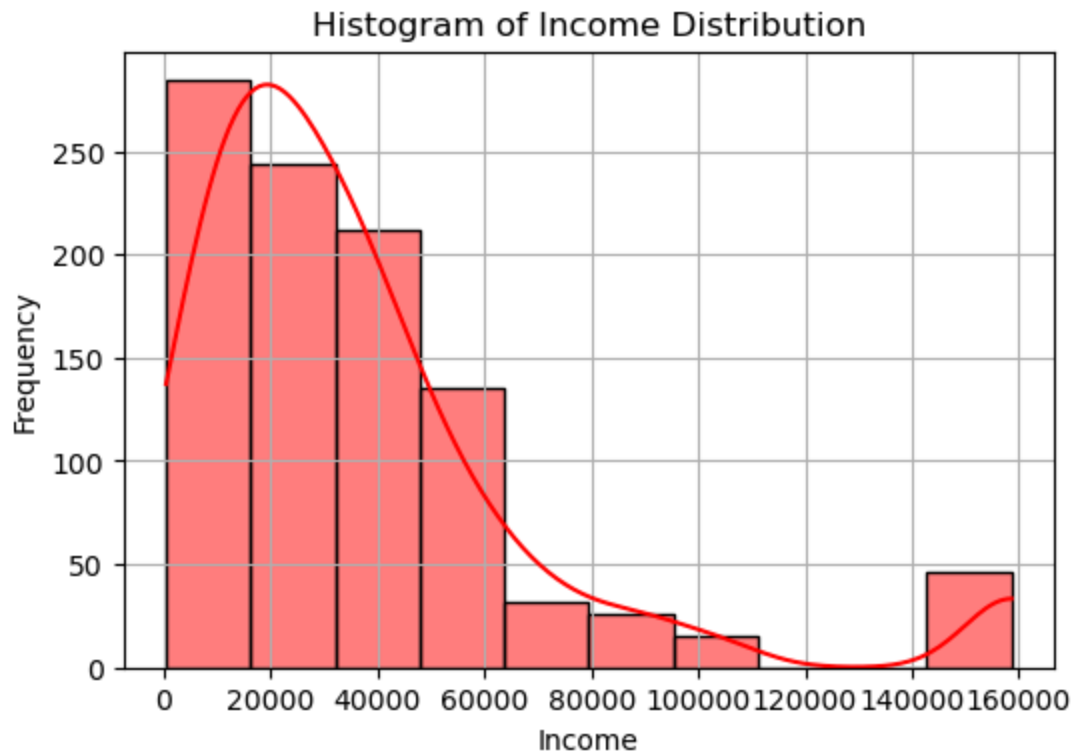
```

```
sns.histplot(data=df, x="income", bins=10, kde=True, color="red")

# Add descriptive labels and title to the plot
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.title("Histogram of Income Distribution")

# Add grid lines for better readability of values
plt.grid(True)

# Display the plot
plt.show()
```



Box plot of Income

```
In [17]: import matplotlib.pyplot as plt
import seaborn as sns

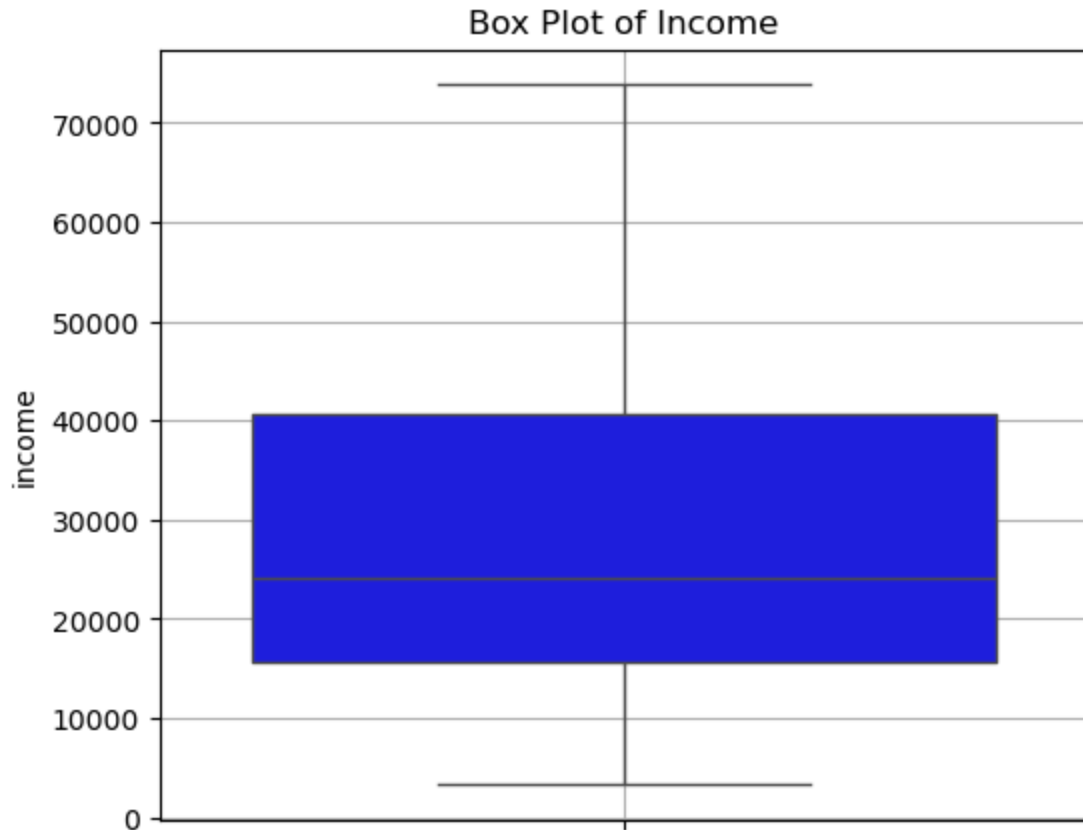
# Set figure size to 6 inches wide by 5 inches tall
plt.figure(figsize=(6, 5))

# Create a box plot showing the distribution of income values in the dataset
sns.boxplot(data=data, y="income", color='blue')

# Add a descriptive title to the plot
plt.title("Box Plot of Income")

# Add grid lines to the plot for better readability of values
plt.grid(True)
```

```
# Display the plot  
plt.show()
```



Explanation:

- `sns.boxplot(y=df['income'], color='lightblue')`: Creates a vertical box plot with a blue color.
- `plt.title()`: Adds a title to the plot.
- `plt.show()`: Displays the plot.

The box plot helps visualize:

- ✓ Median (central line inside the box)
- ✓ Interquartile Range (IQR) (box boundaries)
- ✓ Outliers (points outside the whiskers)

Box Plot of Income Across Degree

```
In [18]: import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Set figure size for better visualization  
plt.figure(figsize=(8, 5))
```

```

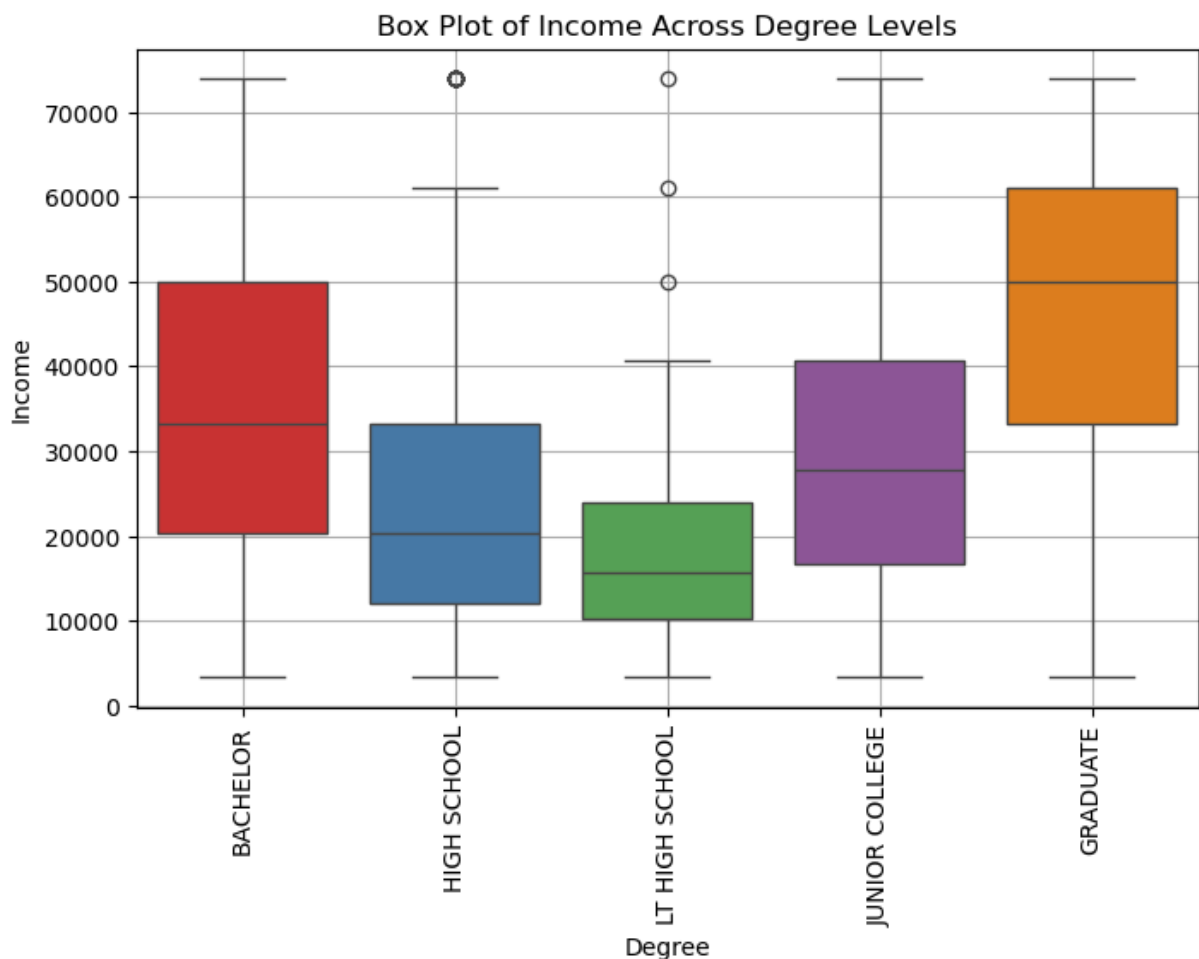
# Create box plot using seaborn
# This commented line shows an alternative way to create the box plot without the '
# sns.boxplot(data=data, x="degree", y='income', palette="Set1")
# Create box plot with 'hue' parameter set to "degree" to color boxes by degree cat
sns.boxplot(data=data, x="degree", y="income", hue="degree", palette="Set1", legend

# Add descriptive labels and title to the plot
plt.xlabel("Degree")
plt.ylabel("Income")
plt.title("Box Plot of Income Across Degree Levels")

# Rotate x-axis labels by 90 degrees to prevent overlap and improve readability
plt.xticks(rotation=90)

# Add grid lines to the plot for easier value comparison
plt.grid(True)
# Display the final plot
plt.show()

```



Scatter Plot of Age and Income

```

In [19]: import matplotlib.pyplot as plt
import seaborn as sns

# Set figure size to 8 inches width by 5 inches height

```

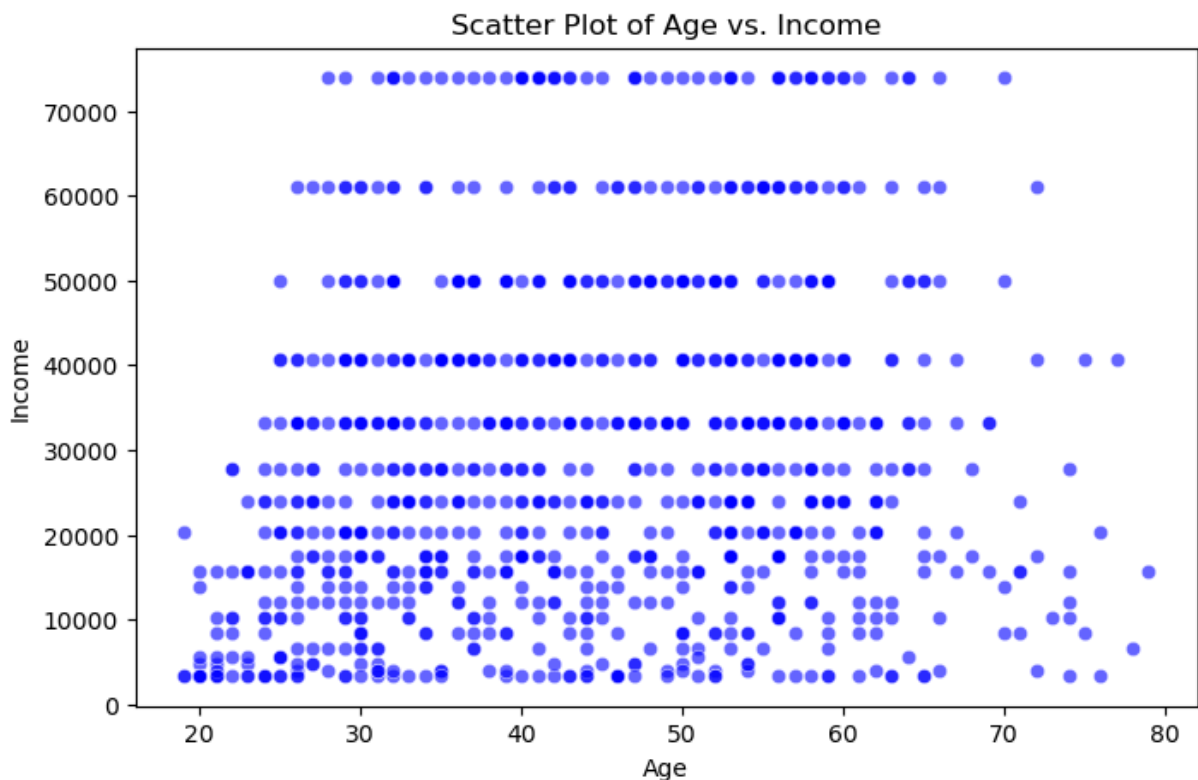


```
plt.figure(figsize=(8, 5))

# Create scatter plot using seaborn with age on x-axis and income on y-axis
# Alpha of 0.6 makes points semi-transparent to better visualize overlapping points
sns.scatterplot(data=data, x='age', y='income', color='blue', alpha=0.6)

# Add descriptive labels for the x and y axes
plt.xlabel("Age")
plt.ylabel("Income")
# Add a title to describe the visualization
plt.title("Scatter Plot of Age vs. Income")

# Display the plot on screen
plt.show()
```



Explanation

- `sns.scatterplot(x=df['age'], y=df['income'], color='blue', alpha=0.6)`: Creates a scatter plot where age is on the x-axis and income is on the y-axis.
- `alpha=0.6`: Adds transparency to the points for better visibility in dense areas.
- `plt.xlabel()`, `plt.ylabel()`, `plt.title()`: Labels and title for clarity.

This scatter plot helps visualize:

- ✓ Income trends across age groups
- ✓ Possible relationships or patterns

✓ Potential outliers

Add the Line of Best Fit to the Scatter Plot

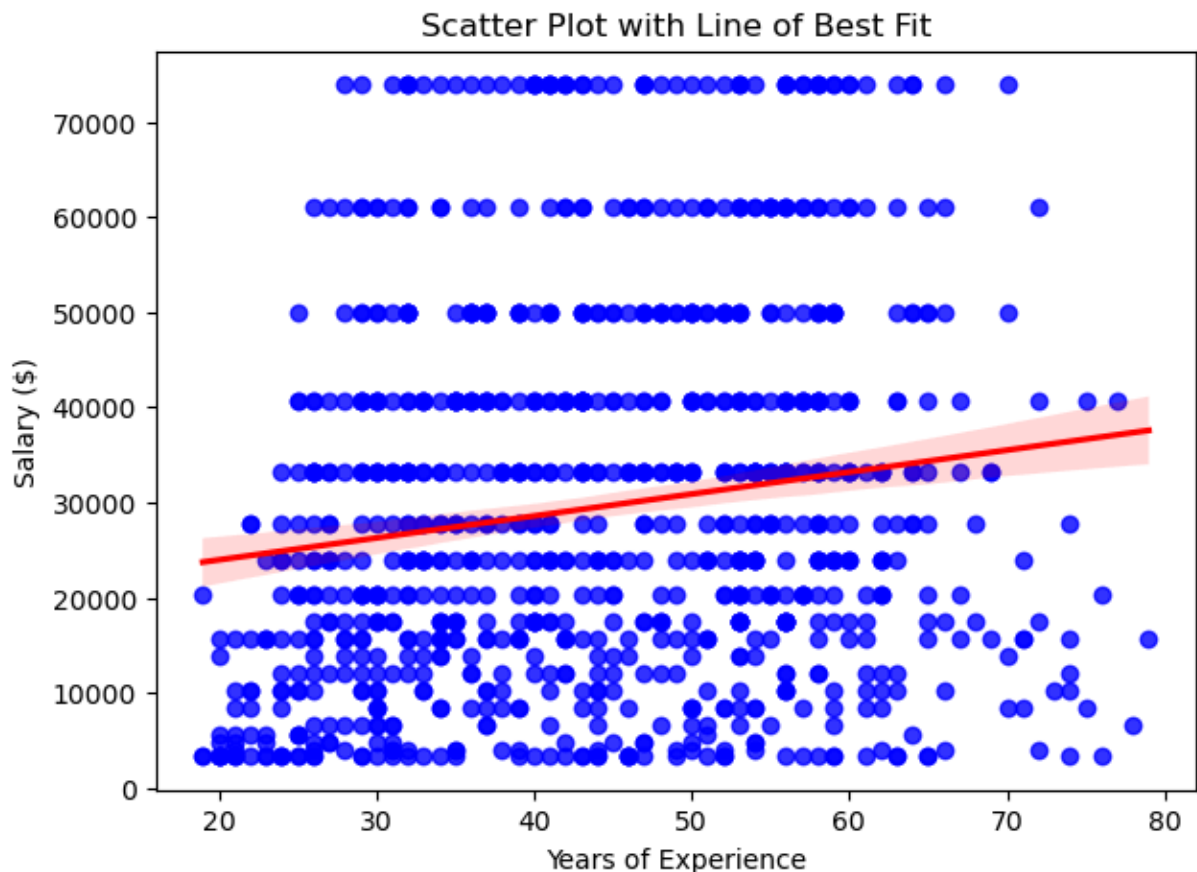
```
In [20]: #install library scipy
        #!pip install scipy
```

```
In [21]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from scipy.stats import linregress

# Create a scatter plot with a regression line
# - figsize=(7, 5) sets the figure dimensions to 7x5 inches
# - regplot creates scatter plot with regression line in one command
# - scatter_kws and line_kws customize the appearance of points and line
plt.figure(figsize=(7, 5))
sns.regplot(x="age", y="income", data=data, scatter_kws={'color': 'blue'}, line_kws=

# Customize plot with title and axis labels
plt.title("Scatter Plot with Line of Best Fit")
plt.xlabel("Years of Experience")
plt.ylabel("Salary ($)")

# Display the plot
plt.show()
```



```

In [22]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from scipy.stats import linregress

# Remove missing values to avoid 'nan' issues in calculations
df_clean = df[['age', 'income']].dropna()

# Perform linear regression to find the relationship between age and income
# Returns: slope (coefficient), intercept, r-value (correlation coefficient),
# p-value (statistical significance), and standard error
slope, intercept, r_value, p_value, std_err = linregress(df_clean['age'], df_clean[

# Create a figure with specified dimensions (width=8, height=5 inches)
plt.figure(figsize=(8, 5))

# Create scatter plot of raw data points
# alpha=0.6 makes points semi-transparent to better visualize overlapping points
sns.scatterplot(x=df_clean['age'], y=df_clean['income'], color='blue', alpha=0.6, 1

# Generate x-values across the range of ages in the dataset
x_vals = np.linspace(df_clean['age'].min(), df_clean['age'].max(), 100)
# Calculate corresponding y-values using the regression equation:  $y = mx + b$ 
y_vals = slope * x_vals + intercept
# Plot the regression line
plt.plot(x_vals, y_vals, color='red', label="Best Fit Line", linewidth=2)

# Format the equation text to display on the plot
equation_text = f"y = {slope:.2f}x + {intercept:.2f}"

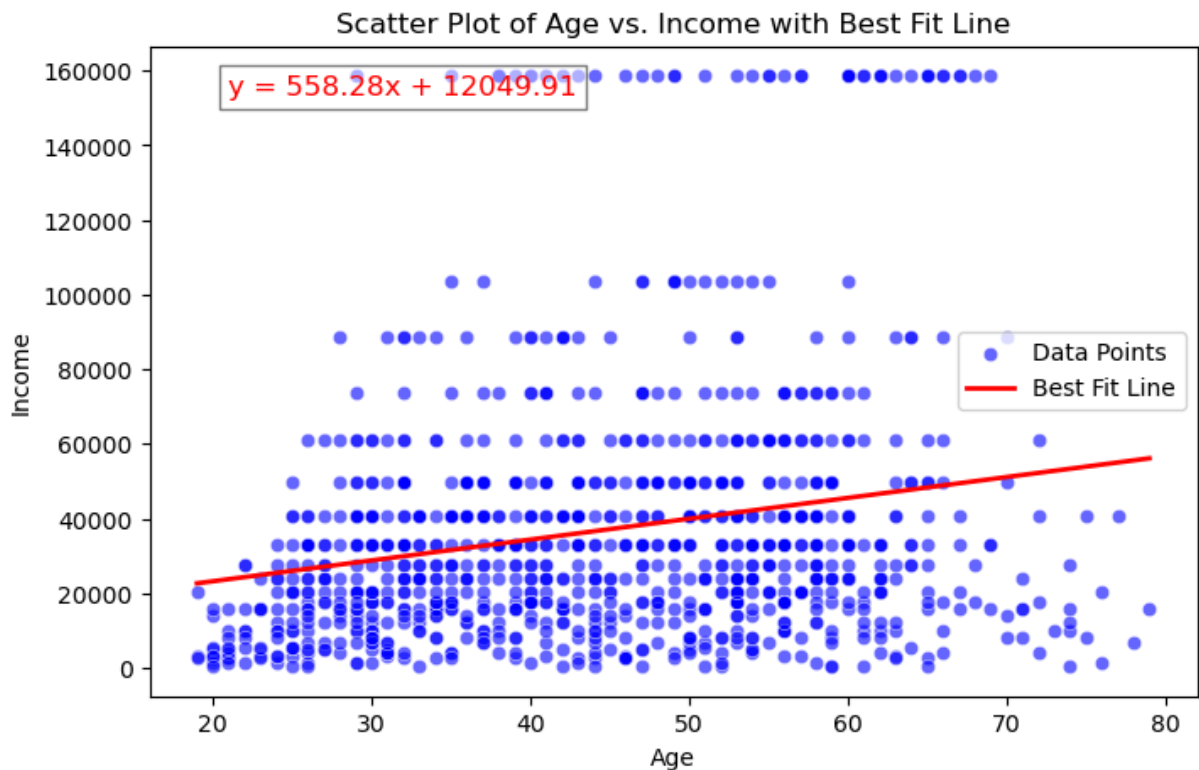
# Add the equation text to the plot with a white semi-transparent background
# Positioned near the top-left of the plot (2 units right of min age, 5000 units be
plt.text(df_clean['age'].min() + 2, df_clean['income'].max() - 5000, equation_text,
        fontsize=12, color='red', bbox=dict(facecolor='white', alpha=0.5))

# Add axis labels and title for clarity
plt.xlabel("Age")
plt.ylabel("Income")
plt.title("Scatter Plot of Age vs. Income with Best Fit Line")

# Display the legend to identify scatter points and regression line
plt.legend()

# Render the plot
plt.show()

```



Homework

- Import a Dataset :
- Download a dataset (e.g., from Kaggle or UCI Machine Learning Repository).
- Import the dataset into R using `read.csv()` or `read_excel()`.
- Clean the Dataset :
- Handle missing values by either removing them or imputing with the mean/median.
- Detect and treat outliers using the IQR method.
- Perform Exploratory Data Analysis (EDA) :
- Calculate summary statistics (mean, median, variance, standard deviation).
- Create visualizations (histograms, boxplots, scatterplots) to explore relationships in the data.
- Estimate the Missing Values for the Categorical Variable



CHUKA UNIVERSITY
Knowledge is Wealth (Akiti ni Mali)
(Sapientia Divitia Est)

**CENTER FOR DATA ANALYTICS
AND MODELING**

ARTIFICIAL INTELLIGENCE | DATA SCIENCE | CONSULTANCY | RESEARCH HUB



SERVICES OFFERED

- Short Course Training in Data Science
- Research (Leveraging AI Tools)
- Consultancy

SHORT COURSE

- R & Rstudio
- Python
- Power BI
- STATA, SPSS & others

Phone : 020-2310512/18
Email: info@chuka.ac.ke

[ChukaUniversityOfficialPage](#) [ChukaUniversityOfficial](#) [@chuka_uni](#)
[Chuka University TV](#) [Chuka_University_Official](#) [www.chuka.ac.ke](#)

<https://cdam.chuka.ac.ke>

Thank you

